

architecture, provides powerful analytical capabilities (<http://www.vertica.com/resources/white-papers>).

There have been several commercial and open source RDBMS products, including IBM's DB2, Oracle Database, Microsoft SQL Server, MySQL and many others. Historically, relational databases have been providing transaction processing with the clarity that emerged from their formal mathematical models, and an elegant way of storing/retrieving data using SQL. When the reality of planetary-scale data management kicked in with the Big Data explosion, combined with the need for massive Web capabilities fuelled by Web 2.0, the industry felt the need for alternatives to traditional RDBMS. Implementations of RDBMSs are generally tuned for their specific category of usage—say, OLTP, data warehousing, decision-support regimes, etc. Big Data applications are characterised by mostly read accesses, need for quick retrieval of query results even if the results are not fully complete, streaming media requests, textual search in large number of documents, 24X7 Web requests with varying data access patterns. However, RDBMS are not tuned for such big data application characteristics.

Architectural shifts on data-sharing models

A number of techniques have been proposed to address the changing needs of data management driven by Big Data and the Cloud. These include:

- a) Data replication, which creates multiple copies of the databases. The copies can be read-only, with one master copy where updates occur, and then are propagated to the copies—or the copies can be read-write, which imposes the complexity of ensuring the consistency of the multiple copies.
- b) Memory caching of frequently accessed data, as popularised by the memcached architecture.
- c) From the traditional 'Shared Everything Scale-up' architecture, the focus shifted to 'Shared Nothing Scale-up' architectures. The shared-nothing architecture allows independent nodes as the building blocks, with information replicated, maintained and accessed. Database sharding is a method of horizontal partitioning in a database, which typically partitions its data among many nodes on different databases, with replication of the application's data via synchronisation. Shared-disk clustered databases, such as Oracle RAC, use a different model to achieve scalability, based on a 'shared-everything' architecture that relies upon high-speed connections between servers. The dynamic scalability required for cloud database offerings still remains elusive in both these approaches. 'Shared-nothing' architectures require time-consuming and disturbing data rebalancing when nodes are added/deleted. While node addition/deletion is faster in the 'Shared-everything' architecture, they have scaling issues with increasing node counts.

Note that using the above techniques seriously impacts the maintenance of the ACID properties satisfied by traditional RDBMS engines. For instance, maintaining multiple read/write copies of the database impacts data consistency. Data sharding impacts atomicity requirements. Schema changes are time-consuming, and need to be propagated to multiple nodes in such data replicated/sharded/partitioned architectures. Various SQL operations, such as joins, cannot be implemented at the database layer, since the database is partitioned; they need to be implemented in the application middleware layer. Therefore, supporting both RDBMS and distributed databases, which can scale to the needs of Big Data and the Cloud, will have conflicting requirements.

Concurrency control

Relational data stores implement pessimistic concurrency, implying that updates to the records are protected and serialised. The locking overhead associated with the pessimistic concurrency control presents significant challenges for any light-weight implementation that needs to meet the high-performance requirements for massive data-stores. On the contrary, many of the non-relational database implementations lean towards optimistic concurrency control, with a relaxed consistency model.

Shifts in the programming paradigm

The non-relational data models allow one to massively scale and perform in parallel. Map-Reduce (MR) techniques allow processing of partitioned sub-problems in a distributed manner, with a greater degree of efficiency. There are libraries available in several forms that allow distributed processing of the map and reduction operations, with map operations executed in parallel. Map-Reduce functions are both defined as key/value pairs, largely capable of operating on top of NoSQL data stores. It also provides inherent fault tolerance to re-compute partial map results.

When does a static schema not make sense? Or, when does it need to be dynamic? A database schema refers to the organisation of data to create a blueprint of how a database will be constructed. It specifies, based on the database administrator's knowledge of possible applications, the facts that can enter the database, or those of interest to the possible end-users. There is an incredible amount of diversity in terms of how data is structured in the Big Data era. The requirements for many data collections are different, and it could be a daunting task to attempt to unify all types of collections with a single schema. Instead of creating a 'one size fits all' approach at the database level, the administrators who use this approach often look for flexibility in schemas. With the inability to design a predetermined schema during the design phase, a traditional SQL-based relational database is less likely to be appropriate.

The widely varying requirements of data management under Big Data and cloud computing resulted in the industry